# CeTTa:
## A Direct C Runtime for MeTTa with Verified Translation

Zar Goertzel
Oruži

July 2, 2026

**Abstract**

MeTTa is a reflective, non-deterministic meta-language with three actively maintained runtimes: Hyperon Experimental (HE, Rust), PeTTa (Prolog), and CeTTa (C). This paper describes CeTTa, a direct C implementation of the HE MeTTa evaluator that passes a 535-test MeTTa regression suite (a 539-row manifest): 405 passed / 0 failed on the default build and 401 passed / 0 failed on the MORK-bridge build, plus a heavy-golden lane at 17 passed / 0 failed (suite snapshot 2026-07-02, commit 9237afd), backed by a 333-case rule-indexed conformance catalog against upstream HE v0.2.10 covering 24/24 specification rule capacities (catalog dated 2026-06). On cross-runtime benchmarks—genomic-inference rows re-receipted 2026-07-02 on commit 9237afd, branching-search rows measured 2026-06-27—CeTTa is fastest on startup-dominated shallow inference and on several genomic-inference rows (the 183-pair PLN evidence core, the full 1,399,997-line WM-PLN raw-loader checkpoint, and the 1,411,430-hypothesis deep genomic backward-chaining row), while PeTTa wins on pure branching search (the 85,013-proof Metamath depth-5 benchmark) and on direct 8-Queens search. A rerun-backed cross-engine chaining suite built on community chainers (Geisweiller's dependent-typed chainers and Metamath proofs; Treutlein's PeTTaChainer problems) shows that CeTTa's shallow advantage is startup-dominated rather than a general scaling property. Historical rows whose current drivers now disagree or fail—the old cross-engine miner speed table, the old full-1.4M WM-PLN scaling headline, and BTree—are explicitly excluded from the revised benchmark ladder. The earlier tilepuzzle regression was root-caused to unbounded growth of the evaluation arena and closed by a tail-safe evaluation-arena garbage collector (on by default), restoring tilepuzzle as benchmark evidence: the full 181,440-state 8-puzzle search now completes in 8.80 s at 233 MiB peak resident memory (commit 9237afd). The runtime is organized as an explicit profile family: `he-compat` mirrors Rust HE literally (including its quirks); the principled `he` line carries exact rationals, true division, and per-case recorded divergences; `he-extended` adds labeled extensions; `he-prime` adds dependent telescopes. The one-step reduction relation is reflected into MeTTa itself as a modal surface (`transitions`, `box`, `diamond`, normal-form tests) aligned with the OSLF LanguageDef → ReductionSpan → $\Diamond/\Box$ pipeline proven in Lean, and the first rule classes (grounded dispatch, argument congruence, equation matching) are welded to a reflectable small-step rule table whose rule-removal tests prove the engine genuinely depends on the declared rules. The runtime features SymbolId-based dispatch (38% instruction reduction), structured space kinds (queue, hash, stack), a pluggable match backend with PathMap/MORK integration, and compiled space artifacts via MORK's `.act` format for zero-copy indexed loading. It is backed by a sorry-free Lean proof (5,400+ lines) establishing semantic correspondence between HE and PeTTa evaluation on the pure fragment, including import commutativity and operational bridges for state and space allocation. A kernel-checked HE′ telescope formalization extends the type system with dependent binder semantics. The RhoCalc reaction machine's eval-at-COMM semantics is given a witness-free, axiom-clean Lean characterization: the quiescent run of the canonical payload-evaluation process equals its structurally observable outcome

set. We present a benchmark ladder from shallow evaluation through large-scale genomic inference and a cross-engine chaining suite, a freshness-first performance re-audit and triage, a specification gap analysis, and a three-machine architecture (local search, indexed shared-space, type elaboration) informed by GPT-5.4 Pro architectural review.

# 1 Introduction

MeTTa [1] is a reflective, typed, non-deterministic language designed as the core language of the OpenCog Hyperon AGI framework. Three implementations exist:

**HE** Hyperon Experimental (Rust, v0.2.10). The reference implementation with Python bindings and a large standard library.

**PeTTa** Prolog-based MeTTa (SWI-Prolog). Leverages Prolog's SLD resolution for backward chaining and pattern matching.

**CeTTa** Direct C evaluator (this work). Targets the HE spec with arena allocation, discrimination-trie indexing, and tail-call optimization.

**Contributions.**

1. A direct C runtime passing a 535-test MeTTa regression suite (539-row manifest; default build 405/0, MORK-bridge build 401/0, heavy-golden lane 17/0) plus 112 profile tests, validated against upstream HE v0.2.10 as oracle.

2. An explicit profile family (`he-compat` / `he` / `he-extended` / `he-prime`) with per-case recorded divergences and a fail-by-default conformance harness: a 333-case rule-indexed catalog (all agree, 24/24 spec-rule capacities) and a 378-example broad corpus with every difference classified (Section 3).

3. A reflective modal LTS surface over the evaluator's one-step reduction relation—successor set, $\Box/\Diamond$, normal-form tests—with runtime De Morgan self-tests mirroring the Galois connection proven in the Lean OSLF framework.

4. The first specification weld: a reflectable small-step rule table gating grounded dispatch, argument congruence, and equation matching, with anti-decorative rule-removal tests. This converts a cross-branch one-step soundness bug (missing argument congruence) into a structurally prevented class.

5. A bidirectional HE$\leftrightarrow$PeTTa translator (Prolog + executable Lean), with a sorry-free Lean proof of semantic correspondence.

6. Cross-runtime benchmarks (genomic-inference rows re-receipted 2026-07-02 on commit 9237afd; branching-search rows measured 2026-06-27, pre-GC): CeTTa is 10–100$\times$ faster than HE on shallow evaluation, wins several current genomic-inference rows (including the full TFBS-inclusive WM-PLN raw-loader checkpoint), while PeTTa is 4.8$\times$ faster on the audited depth-5 Metamath proof-search row and faster on direct 8-Queens search; only still-unresolved rows are quarantined from the revised ladder.

7. A tail-safe evaluation-arena garbage collector, on by default, differentially validated (not verified) against the collector-off reference across the fast regression corpus (406/406 agree) and under ASan/UBSan with provenance assertions (404/404 agree); its headline effect is the restored tilepuzzle capacity row—the full 181,440-state search completes in 8.80 s at 233 MiB, versus unbounded arena growth before (Section 7.2).

8. A characterization of the semantic boundary between HE and PeTTa: the shared fragment (85% of programs) runs identically; the divergence is in free-variable scoping across non-deterministic branches.

9. A specification gap analysis of non-determinism plus mutable state, with 8 concrete examples showing that sequential-per-iteration semantics (CeTTa's default) is correct in 6/8 cases, while HE's interleaved side-effect evaluation loses atoms in real code (the pattern miner).

10. `with-space-snapshot`: a CeTTa extension for branch-isolated mutation, enabling semi-naive forward chaining and speculative search without the side-effect hazards of shared mutable state.

11. A witness-free, axiom-clean Lean characterization of the RhoCalc reaction machine's eval-at-COMM semantics: the quiescent run of the canonical payload-evaluation process equals its structural-congruence-observed outcome set, with no certification witness in the carrier (Section 5.4).

# 2 CeTTa Architecture

CeTTa is a direct evaluator, not a compiler or boot pipeline. It implements the six mutual functions of the HE evaluation spec (`EvalAtom`, `MettaCall`, `InterpretExpression`, `InterpretFunction`, `InterpretArgs`, `InterpretTuple`) as C functions with a shared arena allocator.

## 2.1 Core Design

**Arena allocation.** All atoms are allocated from 64 KB arena blocks. No per-atom `malloc`/`free`. Batch deallocation at scope exit.

**Symbol interning.** A hash table maps symbol strings to unique pointers, enabling O(1) pointer comparison for symbol equality.

**Equation index.** Equations `(= lhs rhs)` are indexed by head-symbol hash into 256 buckets, with a discrimination trie within each bucket for fast LHS matching.

**Substitution tree.** An epoch-based discrimination tree (`SubstTree`) eliminates per-candidate variable renaming during equation queries by tagging indexed variables with monotonic epochs.

**Tail-call optimization.** A `TAIL_REENTER` trampoline with `goto tail_call` eliminates stack growth for recursive equation dispatch.

**Unlimited fuel.** Default fuel is $-1$ (unlimited), matching the HE spec. Stack overflow is caught via `sigaltstack` + `SIGSEGV` handler with a clean error message.

**Two-stage stdlib.** The standard library is precompiled into a binary blob at build time (`cetta-stage0` → `-compile-stdlib` → blob → `cetta`), giving 6 ms startup.

### 2.2 Test Suite

CeTTa is validated against the HE CLI (v0.2.10) as oracle. Golden regression files are oracle-validated when introduced; `make test` replays them on every build. As of the 2026-07-02 snapshot on commit 9237afd: **405 passed, 0 failed, 71 skipped** on the default build and **401 passed, 0 failed, 75 skipped** on the MORK-bridge build (all bridge sub-lanes green), plus 112 profile tests covering module imports, foreign (Python) function dispatch, numeric-tower and profile-surface splits, and extension surfaces. Two further lanes are fail-by-default: `make test-step-rules` disables individual rule-table entries and *requires* the witnesses to fail (Section 3), and the semantic conformance suite re-runs a 23-case subset of the curated oracle corpus live against upstream HE (Section 3.2).

## 3 Specification-Driven Semantics: Profiles, Conformance, and the Small-Step Rule Table

CeTTa is organized around an explicit principle: *the semantics is a named object, not an emergent property of the implementation.* This section describes the profile family, the oracle-conformance campaign against upstream HE, the evaluator bug classes that campaign exposed, the reflective modal surface over the one-step reduction relation, and the first welded rule pack that makes the reduction rules themselves runtime data.

### 3.1 The Profile Family

Profiles are distinct semantic products, implemented as additive layers—never as visibility masks over one shared universe:

**`he-compat`** A literal mirror of Rust HE v0.2.10, including its quirks: integer division for `/`, the strict three-argument `if` surface, Rust's library inventory behavior. Compatibility means matching upstream *behavior*, not reproducing its library set; library availability is out of scope for conformance.

**`he`** The principled line. Spec-faithful where the written specification speaks; where HE's implementation and its specification diverge, the divergence is adjudicated and recorded per-case. Carries exact arbitrary-precision rationals, true division for `/`, floor division `//`, Euclidean `math:mod`, and the clean `math:` namespace over the Rust-heritage `*-math` ops. Extensions that accept more programs (e.g. two-argument `if`) are deliberate, recorded surface choices.

**`he-extended`** `he` plus labeled CeTTa extensions: typed space kinds, `with-space-snapshot`, the `lts` reflection surface, MORK/PathMap-backed spaces.

**`he-prime`** The dependent-telescope extension (Section 10).

Two rules keep this family honest. First, unknown symbols stay inert: MeTTa's philosophy is to leave uninterpreted what it does not know how to interpret, so no profile ever raises an "unsupported feature" error—a profile determines what is *interpreted*, never what is *legal*. Second, profile differences are explicit splits with named tests (e.g. `profile_if_compat_arity` pins that `he-compat` rejects two-argument `if` exactly as upstream does, while `he`/`he-extended` accept it as a recorded extension).

### 3.2 Oracle Conformance

For `he-compat`, upstream HE v0.2.10 is the sole oracle, and the harness is a fail-by-default gate, not a green-manufacturing report. Comparison is by *result-bag equality*—multisets of parsed results, never byte-equality of output text—so formatting differences cannot masquerade as semantic agreement, and duplicate results (which are semantically real in MeTTa) are counted.

**Curated catalog.** 333 cases, every one oracle-validated and in agreement, each mapped to the specification rules it exercises: 24/24 spec-rule capacities are covered, and every capacity is classified into one of three lanes—*reduction-span* (the one-step semantics the modal layer consumes), *supporting-surface* (match, unify, grounded dispatch plumbing), and *spatial-policy* (module import, inventory, space policy). The lane split keeps policy surfaces out of the reduction relation. This full rule-indexed sweep is a periodically refreshed artifact (2026-06); the per-build oracle gate (`make test-he-compat-semantic-suite`) re-runs a 23-case subset of it live against upstream HE on every build.

**Broad corpus.** 378 upstream examples: 302 agree, 59 classified differences (library availability, which is out of conformance scope; path-import capabilities upstream rejects; environment and rendering differences; and adjudicated upstream drift), 7 timeouts, 10 not runnable upstream. No difference is left unclassified.

**Adjudicated divergences.** Where CeTTa deliberately diverges from upstream, each case carries independent grounds. Three examples: upstream's post-0.2.10 `function/NoReturn` behavior contradicts its own written specification (a deliberate loop fix that left the document stale); upstream's `get-doc` errors on HE's own documentation example (an arity regression); and the user-visible result of top-level `eval` on a non-reducible term is genuinely under-determined by the specification—both engines deviate from the literal instruction-layer reading, in opposite directions.

The catalog carries an explicit `no_runtime_trace_or_certificate_surface` flag, enforced by the suite: conformance is *testing*, and the specification is what gets *proven about* in Lean. CeTTa does not emit execution traces or certificates, and a per-run check is never presented as a proof.

### 3.3 Evaluator Bug Classes the Campaign Exposed

Three bug classes were found and fixed, all instructive.

**The Empty algebra.** When overlapping equations yield a branch that reduces to `Empty`, two dual defects appeared: the surviving sibling branch was returned without being re-driven to applicative normal form (violating the spec's applicative order), and in other shapes `Empty` itself leaked into the visible result bag (the spec is explicit that `Empty` "is not returned among other results"). The fix treats the evaluator's outcome set as an algebra with `Empty` as its absorbing zero and re-drives surviving branches independently of their siblings. In modal terms (Section 3.4): the successor algebra must satisfy $\Diamond\bot = \bot$.

**The one-step congruence bug.** The reflective one-step surface reported `(+ 1 (+ 2 3))` as a *normal form*: the one-step relation lacked argument congruence (stepping a reducible argument when the head cannot fire), while the big-step evaluator had it. Because the one-step relation and the evaluator were separately hand-maintained C branch sets, the gap replicated identically across

three development branches and was invisible to every test that only exercised head-reducible terms. The bug was diagnosed through the modal surface itself ($\Box\bot$ holding on a reducible term), fixed as deterministic left-to-right congruence matching the evaluator's applicative order—and is the direct motivation for the rule-pack weld in Section 3.5.

**The switch/switch-minimal conflation.** Upstream HE types `switch` as `(-> %Undefined% Expression %Undefined%)`—the scrutinee is *evaluated* before matching—while `switch-minimal` is `(-> Atom Expression Atom)`, matching the raw scrutinee structurally. CeTTa routed both spellings through one structural handler, so `(switch (+ 1 2) ((3 three) (($x $y) (got $x $y))))` returned `(got 1 2)` where upstream returns `three`. The defect was exposed not by testing but by the certification campaign (Section 3.6): modeling the rules in Lean forced a precise reading of the two upstream type contracts, and an oracle probe confirmed the divergence. The fix splits the rule into `HES_Switch` (evaluated scrutinee) and `HES_SwitchMinimal` (structural), with a distinguishing witness pair pinned in the test suite.

### 3.4 The Reflective Modal Surface

The library surface `lts` (generic, step-parameterized) and `lts:he` (bound to the HE evaluator's one-step relation) reflect the one-step reduction relation into MeTTa as data:

- `lts:he:transitions` emits the one-step successor set of an atom as quoted states (zero results on quiescent atoms); `lts:he:trace-2` enumerates two-step traces.
- `lts:he:is-normal-form` is exactly $\Box\bot$ (every successor satisfies falsity, i.e. no successor exists); `lts:he:is-can-step` is $\Diamond\top$.
- `lts:he:box` $\phi$ and `lts:he:diamond` $\phi$ generalize these to arbitrary unary predicates: $\Box\phi$ holds when every one-step successor satisfies $\phi$ (vacuously at normal forms), $\Diamond\phi$ when some successor does.

The De Morgan law $\Box\phi = \neg\Diamond\neg\phi$ is installed as a runtime self-test; it is the executable mirror of the Galois connection proven once and for all in the Mettapedia OSLF framework, where LanguageDef $\to$ RewriteSystem $\to$ ReductionSpan $\to (\Diamond, \Box)$ is derived generically and `langGalois` establishes the adjunction. Native behavioral types in the sense of Native Type Theory arise as predicates over exactly this reduction span; the runtime surface and the Lean derivation describe the same object at two levels.

Multiplicity discipline: `transitions` preserves the result *bag* (duplicate successors are semantically real), while the modal predicates quotient to reachability. Evaluation and conformance compare bags; $\Diamond/\Box$ use support. Result sets are never deduplicated for speed.

### 3.5 The Small-Step Rule Table

The congruence bug demonstrated the structural disease: reduction rules as anonymous, duplicated C branches are invisible, unremovable, and untestable-by-absence. The cure is to make the rule set a *named, reflectable object* the evaluator provably depends on.

Three granularity strata are kept distinct. *HEFine* is the Lean `mettaHE` instruction machine (59 rules over interpreter states; the proof reference). *HE small-step* is the coarse user-visible one-step relation—one observable surface rewrite, e.g. `(+ 1 (+ 2 3))` $\to$ `(+ 1 5)`—which is what the `lts` surface exposes. *HEBigStep* is full evaluation. The table encodes the small-step relation; its

relationship to the fine machine is a collapse/simulation with explicit provenance links (each coarse rule names the fine rules that justify it), never a label identity.

The runtime object, `CettaLangDefPack`, carries: language, profile, and granularity identifiers; a schema version; a source digest over the canonical rule-descriptor text; the rule table; a disabled-rule mask populated from the `CETTA_LANGDEF_DISABLED_RULES` environment variable; and a *per-rule claim level* recording the strongest evidence behind each rule (Section 3.6). Ten rule classes are live and gate the *only* implementations of their behavior in the one-step subsystem: grounded dispatch, leftmost expression congruence, equation match, `eval`, `chain`, `let`, `let*`, `case`, `switch`, and `switch-minimal` (the last two split per their distinct upstream contracts, Section 3.3). Quote/return and empty/error quiescence are structural rows: accounted in the table with written reasons, but not gateable, since disabling them would make the semantics incoherent rather than smaller. Each row carries a provenance string naming the fine-machine rules that justify it. The pack is reflected into MeTTa by `lts:he:step-rules`.

The load-bearing test is *anti-decorative*: a dedicated lane re-runs the witness suite with each live rule disabled and *requires* the witnesses to fail (the successor set must go empty; normal-form must flip)—baseline plus ten rule-removal failures confirmed. If the witnesses still passed with a rule removed, a duplicate hidden code path would exist—precisely the disease that produced the congruence bug. A disabled special form goes *inert* (its head stays uninterpreted; congruence still applies), per the principle that unknown symbols are data, never errors. The lane fails closed in both directions and runs on both the default and MORK-bridge builds.

Claim levels are deliberately *excluded* from the digest: the digest fingerprints rule *content* (names, profiles, provenance, schema), while claims are *evidence about* that content. Upgrading a rule's certification therefore must not—and verifiably does not—move the semantic object: the digest (`fnv1a64:12eec5b655a75904` at the time of writing) stayed fixed across every claim flip. The pack is, by design, a hand-authored object kept honest by the removal lane and the digest; we deliberately parked an earlier plan to generate specialized C from the rule source, because the weld's value lies in the named semantic object and its fail-closed tests, and code generation is machinery the certification program does not need.

### 3.6 Lean Certification of the Rule Table

Each per-rule claim takes one of three values, in increasing strength: *bag-tested-adequate* (the rule's behavior matches the legacy path and oracle bags on the witness corpus), *rule-sound-on-fragment* (a compiled, sorry-free Lean theorem proves the rule absorbed by the official declarative semantics on an explicitly characterized fragment), and *rule-sound* (the same, unconditionally). A claim flips only when the compiled theorem exists, so the reflective surface can never overclaim. The certification route is *absorption*: for each coarse rule, a theorem that whenever the rule relates $a \to a'$, the official declarative judgment of the Mettapedia HE formalization (`MettaCall`, `MinimalStep`, or `EvalAtom`) already relates the same atoms. At the time of writing five of the ten live rules carry certification:

| rule | claim | Lean witness |
|---|---|---|
| `HES_GroundedDispatch` | rule-sound | `mettaCall_absorbs_grounded_dispatch` |
| `HES_EquationMatch` | rule-sound | `mettaCall_absorbs_equation_match` |
| `HES_Eval` | rule-sound | `minimalStep_absorbs_eval` |
| `HES_Chain` | rule-sound-on-fragment | `minimalStep_absorbs_chain_source/_subst` |
| `HES_LeftmostExprCongruence` | rule-sound-on-fragment | `evalAtom_absorbs_congruence_frag` |
| `HES_Let`, `LetStar`, `Case`, `Switch`, `SwitchMinimal` | bag-tested-adequate | in progress |

The certified fragment for the congruence and chain rules is characterized positively: untyped constructor-headed expressions with quiescent spectator arguments and a grounded-dispatch or equation-match redex. Its supporting theory is a short ladder of compiled lemmas: a quiescence domain on which atoms officially self-evaluate, uniqueness of that self-evaluation, transport of official evaluation across a single changed argument, and absorption of inner fragment steps—about which the honest surprises accumulated (e.g. a bare `Error` *symbol* at tuple head becomes error-shaped, so the fragment must exclude it explicitly).

The five remaining rules are upstream *stdlib sugar*: their one-step behavior is the upstream definition of `let`, `let*`, `case`, `switch`, and `switch-minimal` as `unify/chain` programs. Certifying them required a *matcher bridge*: relating the public equation-query surface to the official `matchAtoms/mergeBindings` semantics. The key insight from this tranche was that the real seam was *not* merely a direct one-way-to-two-way matcher comparison. The public HE query surface had historically routed equation lookup through the simplified `simpleMatch` path, while the faithful two-sided matcher already existed separately. We therefore split the bridge into three explicit layers.

First, `DeclMatchSpec.lean` states the author's `match_atoms` semantics declaratively and proves the executable `matchAtoms` helper sound and complete against it (`matchAtoms_sound`, `matchAtoms_complete`). Second, `DeclMergeSpec.lean` does the same for `merge_bindings/add_var_binding/add` proving `mergeBindings_sound` and `mergeBindings_complete`. Third, `QuerySurfaceAgreement.lean` proves a bounded adequacy theorem for the *public* query interface: `queryEquations_ground_two_way_adequate` and its `queryEquationsAgainstVisible`-companion show that, on the ground-query fragment and under the explicit freshening adequacy hypothesis `EquationFresheningAdequate`, the repaired `matchAtoms/mergeBindings` surface and the legacy `simpleMatch`-based scan agree in both directions up to replay fuel.

The earlier singleton-seed factorization theorem remains one ingredient inside this story, but it is no longer the whole story. The decisive insight is the G3/G3b split: equality-threading becomes observable only when the queried atom itself still contains variables, because that is the point at which `Bindings.resolve/Bindings.apply` must honor equalities rather than plain assignments. We therefore bank the ground-query adequacy result as the honest current certification surface, and carry the variable-bearing query case as an explicit next obligation rather than silently pretending the legacy surface was already faithful.

# 4 Verified HE↔PeTTa Translation

## 4.1 Bidirectional Translator

A bidirectional translator converts between HE and PeTTa dialects. It is implemented both as relational Prolog (`he_petta_relational.pl`, with explicit freshness threading) and as computable Lean functions (`translateHE`, `translatePeTTa`).

The translator handles 8 HE→PeTTa rewrite rules (`chain`→`let`, `collapse-bind`→`collapse`, `nop`→`let $fresh X ()`, etc.) and 5 PeTTa→HE rules (`progn`→nested `let`, `prog1`→`let` with result capture, etc.). All 437 real `.metta` files parse and translate successfully.

## 4.2 Sorry-Free Lean Proof

The translation is backed by a **sorry-free, axiom-free** Lean proof across two files totaling 4,979 lines and 102 definitions/theorems:

`HEPeTTaSound.lean` (1,063 lines, 44 theorems.) Proves that HE's equation matching corresponds to PeTTa's `ruleApp` evaluation. The key result:

> `simpleMatch_applyBindings_comm`: if HE's `simpleMatch` succeeds on two `PureTranslatable` atoms, then applying the translated bindings via `heBindingsToOSLF` to the translated pattern recovers the translated target.

This commutation property, combined with the existing `matchPattern_applyBindings_complete` from the OSLF framework, establishes that a PeTTa `matchPattern` witness exists whenever HE's `simpleMatch` succeeds.

`HEPeTTaTranslate.lean` (3,916 lines, 58 theorems.) The executable translator with:

- `translateHE_translatable`: translation preserves `Translatable` (output admits `atomToPattern`).
- `translateHE_preserves_pureTranslatable`: translation preserves the stronger `PureTranslatable` fragment.
- `translatePeTTa_translatable`: same for the reverse direction.
- 9 `#eval` tests matching Prolog output on concrete atoms.
- 6 `rfl` examples kernel-verified by Lean.
- Roundtrip idempotence examples (kernel-verified).

# 5 Reflective $\rho$-calculus and eval-at-comm

Under `-lang rhocalc` the same runtime stops being a term rewriter and becomes a concurrent process engine: its unit of work is not a rewrite but a *reaction*, a rendezvous between a sender and a receiver on a shared channel. This brings channels with private names and genuine concurrency; and because the calculus is *reflective*, a received name can be *run*, so a communication can itself be a computation. The MeTTa library that exposes that last capability is *rhometta*. We build up to it from the pure calculus.

## 5.1 The pure calculus

The calculus is Meredith and Radestock's reflective higher-order $\rho$-calculus [4]. Its one reduction rule, COMM, is a handshake (Figure 1): a sender `x!(m)` on channel `x` beside a receiver `for(y <- x){P}` on that same channel react; the message is consumed and the receiver resumes with `m` substituted for its bound name `y`:

$$x!(m) \mid \mathsf{for}(y \leftarrow x)\{P\} \xrightarrow{\text{COMM}} P[y := m].$$

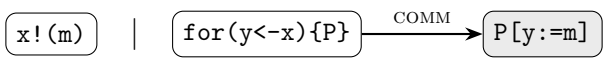


Figure 1: A reaction: a send and a matching receive on channel `x` rendezvous; the receiver continues with the message substituted in.

The rest of the vocabulary is small. `0` is the inert process; `P | Q` runs `P` and `Q` concurrently. Two reflection operators connect processes and names: `@P` *quotes* a process into a name, and `*x`

*drops* (runs) the process a name quotes, with `*@P = P`. Channels are themselves quoted processes — this is what makes the calculus higher-order. CeTTa implements exactly these six strict-core constructors — `rho:nil`, `rho:par`, `rho:send`, `rho:recv`, `rho:quote`, `rho:drop` — with COMM as the only reduction. A program is reduced to quiescence by `cetta -lang rhocalc` *`file`*, written either in a MeTTa-style s-expression surface (`.mrho`) or a Rholang-like surface (`.rho`); the runtime keeps a live index of sends and receive-sites, rebuilt after each reaction.

### 5.2 Reflection is enough

A surprising amount follows from COMM and quote/drop alone, with no further primitives. Each item below is a standalone `-lang rhocalc` program.

- *Name mobility.* A receive that drops its argument, placed beside a send carrying a quoted send, reduces to that inner send made live: a name has travelled across a channel and been activated.
- *Fresh names.* Quoting a distinct process yields a distinct name, so a receiver can mint and use a name it never mentioned syntactically.
- *Replicated input.* An input-guarded server can re-create its own listener: after serving a message it leaves a copy of the receiver in the residual, so the channel keeps serving. Persistent receive is *derived*, not built in.
- *Recursion and divergence.* The same self-recreation, left ungated, runs without bound (the engine reports its reduction limit); recursion needs no dedicated operator.

Private channels and information hiding come along for free: a payload exchanged on a name no other process holds is unobservable apart from whatever completion signal the protocol chooses to publish.

The capstone is *eval-at-COMM*. Let the receiver do nothing but drop what it receives:

$$\texttt{eval\_drop}(x,p) \;\equiv\; \underbrace{x!(@p)}_{\text{send the quoted program } p} \;\mid\; \underbrace{\mathsf{for}(y \leftarrow x)\{\,{*y}\,\}}_{\text{run whatever arrives}}.$$

The COMM delivers the name `@p`; the drop `*@p` runs `p`; the process settles at the result of running `p`. A handshake has become an evaluation step, and since `p` may itself evaluate to more process, a reaction can plant the next reactions.

### 5.3 Profiles

A *profile* names a variant of the reduction relation together with the grammar it accepts. `strict-core` is the pure asynchronous spine above: it admits only the six core constructors, and a non-core form is *rejected* rather than left inert. The `cost` profile accepts a richer resource-annotated dialect and meters each reaction, charging a budget per COMM (Section 5.7).

### 5.4 Certified reduction

Does eval-at-COMM report the result of running the payload, or could the machinery quietly change the answer? We settle this in Lean. Run `eval_drop(x,p)` to quiescence and collect every outcome observable *up to structural rearrangement* of the process; that set is exactly the results a correct evaluation of `p` would produce, and the statement refers only to what is observable,

never to an internal "this is the answer" tag. The theorem `rhometta_bridge_scObserved` (in `RhometaReduction.lean`) equates the concrete quiescent run of `eval_drop(x,p)` with `RhometaSCObservedOutcomes` of the same process — the outcomes visible up to structural congruence — with no result-certificate carried in the answer. Two ingredients are load-bearing. First, a complete canonical-form normalizer `nfAtom`: two processes are structurally congruent exactly when their normal forms coincide, which unblocks inductions that otherwise stall on the symmetry and transitivity of congruence. Second, one hygiene side condition `noBoundUnderQuote`: the communicated variable must not occur under a quote, since a payload that hides a bound occurrence inside a quoted name defeats the characterization (a concrete counterexample exhibits exactly that failure). The development carries no `sorry`. A separate caveat on the trusted base: its conformance and DSL fixtures use `native_decide`; the bridge theorem's own module does not.

### 5.5 rhometta: MeTTa at the reaction

eval-at-COMM becomes a way to program once the payload is a MeTTa term. The library `lib/rhometta.metta` provides `rhometta:eval`, which defers a MeTTa term as the payload to be run at COMM; `rhometta:run`, which reduces a process to its quiescent outcome set; and `rhometta:transitions`, which returns one-step successors. It sits on `lib/rho.metta`, the strict-core constructor surface, and delegates all reduction to the engine. One consequence is central: when a deferred payload evaluates to *several* results, the COMM forks — one successor per result. Nondeterministic computation becomes a branching reaction, and `rhometta:run` returns one residual per branch (the *may-set*).

### 5.6 What reactions compute

**The minimal cell.** The smallest rhometta program is a single cell: a send carrying a deferred term beside a receiver that drops (runs) it. `rhometta-eval-cell` sends `(+ 2 3)` this way and the reaction leaves `5`; with a multi-result payload `(superpose (a b c))` the COMM forks into one branch per result, and `rhometta:transitions` confirms the fork is a single reduction step with three successors. Communication does computation, and result-multiplicity becomes branching.

**A self-unfolding prover.** Seed a *single* cell for a goal. Each COMM evaluates one backward-chaining step whose result is the cells for that goal's subgoals, so `rhometta:run` keeps reducing and the reaction network unfolds itself into a proof tree. Where a goal has several rules the payload has several results, so the COMM forks: `rhometta:run` returns one quiescent residual per complete proof tree (Figure 2) — two for the sample "can I bake a cake?" goal, while a single-rule goal stays confluent at one.


Figure 2: The self-unfolding prover's may-set for "cake": two complete proof trees, one per rule for `batter`. The trees are not enumerated up front — they are *grown* by reactions whose payloads plant their children.

**Type inference as proof search.** A typing judgment `(: proof (HasType e t))` is proved by reactions over typing rules, and the proof term is the derivation. In `rhometta-type-inference`, an overloaded `plus` yields a genuine may-set — an `Int` and a `Float` typing of `(plus one one)` — while `(plus x one)` with `x : Int` prunes the `Float` branch at the unification join. Proof terms stay pure, `(: (app pf pa pb) (HasType e t))` — a typing is crisp, so no truth value is attached. Each result is cross-checked differentially against a sequential chainer over the same knowledge base, which derives the same proof-term skeletons.

**A truth value that is earned.** Crisp proof terms are the right output for a typing, but some judgments are genuinely probabilistic, and there a reaction can carry evidence rather than certainty. In `rhometta-evidence-revision`, two agents each count their own observations of "ravens are black" into an opinion — a strength and a confidence — computed at COMM by a deferred payload. The two agents share two observations, so merging their opinions *naively* double-counts the shared evidence and overstates the confidence (0.889); an overlap-corrected revision discounts the shared stamps for the honest value (0.857), at the same strength. The revision can run after the parallel fan-in, or *at* the final rendezvous itself. This is the one place a truth value belongs — it is counted from data, not attached by hand.

**Composition.** These pieces compose: a further program runs an independent proof search, an overlap-corrected evidence revision, and an attention-spreading step concurrently, then rendezvouses their channel-gathered results into a single synthesis.

### 5.7 Cost-accounted reactions

Reactions are also the natural place to *meter* computation: each COMM is one unit of work, so a resource budget can be charged per reaction. CeTTa's `cost` profile does this directly — it reduces an annotated $\rho$ dialect and reports the resource each reaction consumes. It is the executable end of an ongoing program to equip the $\rho$-calculus with a *cost endofunctor*: a construction that wraps a calculus with resource accounting once and for all [5]. We return to it in future work.

## 6 Cross-Runtime Benchmarks

All benchmarks use dialect-agnostic MeTTa (shared syntax) unless noted. Timing rows in this section were measured on 2026-06-27 (the pre-GC tree), except the grounded genomic-inference rows (PLN evidence core, full WM-PLN raw-loader, mine→chain focus, and deep genomic backward chaining), which were re-receipted on 2026-07-02 on commit 9237afd; rows explicitly described as excluded, methodological, or unresolved carry their own status. The two heavy genomic rows (full WM-PLN raw-loader and deep genomic backward chaining) were re-measured on a quiescent host with the same binary at commit 9237afd; because the two engines are always compared within one session, the cross-engine ratios are the load-bearing claims. For rows whose single-run wall time is below 0.1 s, we report averaged full-process wall time over repeated invocations to avoid timer quantization. Output is verified identical across runtimes whenever a row is presented as a clean cross-runtime benchmark. So a reader can see precisely what is measured, each subsection below opens with the core of the program it runs; the full sources are linked inline and collected at `benchmarks/chaining_crossengine`.

Most of the inference benchmarks are driven by one small dependent-typed *backward chainer*. To prove a goal `(: $proof $theorem)` it either matches a fact in the knowledge base, or—for a

rule application (`$rule $prem`)—looks up the rule's type and recurses on its premise (the binary and $n$-ary cases follow the same shape):

```
(: bc (-> $a Nat $b $b)) ; KB, depth, goal -> goal
(= (bc $kb $_ (: $proof $theorem)) ; base case: it's a fact
   (match $kb (: $proof $theorem) (: $proof $theorem)))
(= (bc $kb (S $d) (: ($rule $prem) $theorem)) ; step: apply a rule
   (let* (((: $rule (-> (: $prem $t) $theorem))
           (bc $kb $d (: $rule (-> (: $prem $t) $theorem))))
         ((: $prem $t) (bc $kb $d (: $prem $t))))
    (: ($rule $prem) $theorem)))
```

Each benchmark below supplies its own typed rules and facts to this chainer (or, where noted, a forward chainer or a truth-value kernel).

### 6.1 Shallow Evaluation

Nil Geisweiller's typed backward chainer (the `bc` above) over 60 ground entities at depth 1 (20 researchers, 20 engineers, 20 artists). The rules are typed binary implications; for example, `r-mortal` combines a researcher fact with a curious fact to derive a mortal witness:

```
(: r-mortal (-> (: $r (researcher $x))
                (: $c (curious $x))
                (mortal $x)))
!(bc &kb (fromNumber 1) (: $proof (mortal $x))) ; -> 60 witnesses
```

All three runtimes produce identical 60 proof-carrying witnesses, e.g. `(: (r-mortal r01 r01c) (mortal r01))`.

| Runtime | Time | vs. CeTTa | Output |
|---|---|---|---|
| CeTTa | 4.0 ms avg | 1× | 60 witnesses |
| PeTTa | 45.4 ms avg | 11.3× slower | 60 witnesses |
| HE | 402 ms avg | 100× slower | 60 witnesses |

This is a startup-dominated sanity row, not a scaling claim. It is useful because it exercises the typed proof-carrying surface while remaining small enough that translator and runtime drift are easy to spot.

### 6.2 Deep Proof Search: PeTTa Wins

Metamath proof search at depth 5 (2 axioms, proving $t = t$, 85,013 proof trees) reverses the performance ordering. The knowledge base is Geisweiller's `demo0` (axiom `a1`: equality is right-Euclidean; `a2`: 0 is a right identity; `mp`: modus ponens), and the goal is `t = t` (`01_nilbc/nilbc_th1.metta`; glyphs shown ASCII here):

```
(: a1 (-> (: $t term) (: $r term) (: $s term)
        (-> (= $t $r) (-> (= $t $s) (= $r $s))))) ; right-Euclidean
(: a2 (-> (: $t term) (= (+ $t 0) $t))) ; 0 right-identity
(: mp (-> (: $maj (-> $P $Q)) (: $P wff) (: $Q wff)
        (: $min $P) $Q)) ; modus ponens
!(bc &kbh (fromNumber 5) (: $prf (= t t))) ; prove t = t
```

| Runtime | Time | vs. PeTTa | Status |
|---|---|---|---|
| PeTTa | 7.60 s | 1× | 85,013 proofs |
| CeTTa | 36.42 s | 4.8× slower | 85,013 proofs |
| HE | >120 s | timeout | killed |

PeTTa's Prolog search remains decisively better on deep branching proof search; the current rerun widens the gap to roughly 4.8×. HE still cannot finish within 120 s.

### 6.3 Forward Chaining at Scale

Nil Geisweiller's propositional-calculus forward chainer at depth 4 uses a solution space that is extended in place by forward application of modus ponens:

```
(= (fs-add-to-ss $rb $ss)
   (with-space-snapshot $ss0 $ss
     (match $ss0 (: $x1 $a1)
       (match $ss0 (: $x2 $a2)
         (match $rb (: $f (-> $a1 $a2 $b))
           (add-atom $ss (: ($f $x1 $x2) $b)))))))
!(count-atoms &ss)
```

The current CeTTa rerun of `tests/nil_pc_fc_d4.metta` completes in 100.86 s and reports 8.19M accumulated results after the four forward passes. We do *not* reuse the older "11.7M generated pairs" sentence as benchmark evidence, because the current HE lane does not time out on this driver; it errors immediately (`match expects a space as the first argument`). So the forward row remains a useful workload, but it is not yet paper-ready as a clean cross-runtime benchmark.

### 6.4 Translator Validation

A PeTTa-native benchmark using `prog1` (PeTTa-specific construct) was translated PeTTa→HE and run on CeTTa. The translator converts `prog1` to nested `let` with fresh variables:

```
; PeTTa original:
(= (run-query) (prog1 (bc &kb ...) (println! "done")))
; Translated to HE:
(= (run-query) (let $__tr_result_1 (bc &kb ...)
                (let $__tr_discard_2 (println! "done")
                  $__tr_result_1)))
```

The current source file `tests/bench_backchain_petta_native.metta` produces 30 proof-carrying witnesses on native PeTTa, and the current translated artifact produces the same 30 witnesses on both CeTTa and HE:

| Runtime | Time | vs. CeTTa | Output |
|---|---|---|---|
| CeTTa | 3.7 ms avg | 1× | 30 proofs |
| PeTTa | 44.8 ms avg | 12.1× slower | 30 proofs |
| HE | 319 ms avg | 86× slower | 30 proofs |

This row is a translator benchmark, not just a syntax example: the observable runtime behavior survives translation.

### 6.5 PLN Evidence Aggregation over Genomic KB

To benchmark inference over real data, we run PLN evidence aggregation over the Rejuve genomic knowledge base (chr16 pilot: 41 GWAS SNPs, 183 SNP–gene pairs, 3 evidence channels). Each pair carries eQTL tissue counts (GTEx, 49 tissues), ABC enhancer–gene contact counts, and a regulatory presence flag. The PLN evidence quantale combines channels via revision (`evidence-hplus`), producing a calibrated SimpleTruthValue per pair. The evidence operations are Lean-verified: `power`, `toLogOdds_power_mul`, and `power_power_inv` are proved in `Mettapedia/Logic/BinaryEvidence.lean`.

The core arithmetic is ordinary MeTTa over numbers:

```
(= (Truth_ModusPonens (stv $f1 $c1) (stv $f2 $c2))
   (stv (+ (* $f1 $f2) (* 0.02 (- 1 $f1))) (* $c1 $c2)))
(= (Truth_Revision (stv $f1 $c1) (stv $f2 $c2))
   (let* (($w1 (Truth_c2w $c1)) ($w2 (Truth_c2w $c2)) ($w (+ $w1 $w2))
         ($f (/ (+ (* $w1 $f1) (* $w2 $f2)) $w)) ($c (Truth_w2c $w)))
     (stv (min 1.0 $f) (min 1.0 $c))))
```

The benchmark is dialect-agnostic (shared MeTTa, no translation needed). The portable `benchmarks/bio_pln` evidence-score drivers produce numerically identical 183 scored pairs on CeTTa, PeTTa, and HE:

| Runtime | Time | vs. CeTTa | Output | Status |
|---|---|---|---|---|
| CeTTa | 10.9 ms avg | 1× | 183 bio-scores | identical |
| PeTTa | 58.1 ms avg | 5.4× slower | 183 bio-scores | identical |
| HE | 7.56 s | 696× slower | 183 bio-scores | identical |

Scores carry genuine biological signal: `rs9923732→ENSG00000166455` scores strength 0.7348066298342542 (133/181) and confidence 0.9945054945054945 (181/182), while most pairs score 0.04 (minimal evidence). All three engines reproduce every one of the 183 scores to full numeric precision on the portable drivers; the earlier non-portable driver diverged only because upstream HE integer-divided the evidence ratio to zero and PeTTa left imported evidence-library calls unreduced. Exact three-engine numeric agreement is this row's validation story.

**What we no longer claim here.** Earlier drafts included a synthetic scaling table from 143 to 1.4M atoms. We still do not treat that older synthetic table itself as benchmark evidence in the paper. What *is* now grounded again is the true full raw-loader checkpoint lane: the fresh CeTTa and PeTTa reruns both execute the same TFBS-inclusive 1,399,997-line data tier and agree on the labeled observable contract (`9` seed targets, `9` contextual targets, `85` disease proofs, `34` supported-drug proof witnesses, and `8` unique supported-drug hypotheses). The explicit proof-count vs. hypothesis-count split matters here: the `34` traces are genuinely multiple proof routes collapsing to `8` unique `(gene,disease,drug,tissue)` hypotheses, not a stale counting mistake. Fresh timings on this grounded full row are 16.4 s on CeTTa and 44.5 s on PeTTa (about 2.7× faster). Upstream HE does not currently run this loader shape directly: its `metta` runner rejects the relative-file `import! ./support/...` surface as an illegal module name, so the HE column for this row remains a loader blocker rather than a performance number. The 183-pair core therefore remains the grounded small PLN row, while the full TFBS-inclusive checkpoint is now the grounded large WM-PLN row.

### 6.6 Genomic Pattern Mining

To exercise real algorithmic depth, we run the Hyperon pattern miner [1] on real GTEx eQTL data, discovering surprising co-regulation patterns—pairs of SNPs or genes that share eQTL evidence more often than expected by independence. The miner is run unmodified from github.com/iCog-Labs-Dev/hyperon-miner; its core loop alternates `build-specialization` (extend a candidate pattern by one link) with a `surprisingness` score that keeps only patterns whose support exceeds the independence expectation. The data is extracted from the Rejuve genomic KB (338M Prolog lines, 57 GB) and discretized into categorical attributes (tissue breadth, effect size) for pattern mining.

The current audit does *not* reuse the older cross-engine mining speed table as benchmark evidence, and no current in-tree driver re-emits the earlier specialization-core atom count, so we

do not carry that number forward. We keep the pattern miner as a methodological workload. Its portability boundary in this revision is HE's stateful use of `superpose`: a remove/add pattern that mutates a shared space in the middle of a `superpose` branch:

```
(superpose ((remove-atom $specspace $spec1)
           (add-atom $specspace $spec1)
           (remove-atom $specspace $spec2)
           (add-atom $specspace $spec2)
           (remove-atom $specspace $spec3)
           (add-atom $specspace $spec3)))
```

Current CeTTa preserves all three atoms produced by this pattern, while current HE's interleaved side-effect evaluation drops the third—the same atom-loss hazard analyzed in Section 9 (the pattern miner). So we keep the miner as a portability/debugging result, not as a fresh cross-engine speed table.

### 6.7 Backward Chaining over Mined Patterns

The mined co-regulation patterns serve as typed inference rules for Geisweiller's `nilbc` backward chainer (the same dependent-typed `bc` listed at the start of Section 6), closing the *mine→chain* pipeline. The knowledge base combines:

- 100,000 real GTEx eQTL links (40,484 SNPs × 14,043 genes),
- 13 co-regulation patterns and 7 co-expression patterns (mined, surprisingness $> 0.98$),
- 3 typed PLN rules: *direct-target* (eQTL → target), *coreg-transfer* (co-regulation + target → target), and *coexpr-transfer* (target + co-expression → target).

The three rules are typed implications fed to the `bc` chainer of Section 6; *direct-target* reads an eQTL fact, *coreg-transfer* chains a co-regulation edge onto an existing target:

```
(rule-sig dt (-> (: $e (eqtl $snp $gene)) ; direct-target
                 (target $snp $gene)))
(rule-sig ct (-> (: $e1 (coreg $snp1 $snp2)) ; coreg-transfer
                 (: $e2 (target $snp2 $gene))
                 (target $snp1 $gene)))
!(bc &kb (fromNumber 2) (: $proof (target rs10000544 $gene)))
```

Bridge base cases let `nilbc` consume untyped data atoms (`eqtl-link`, `co-regulate`) directly, avoiding a format conversion pass over the 100K-atom KB.

On the portable `benchmarks/mine_chain_focus` focus-summary drivers, CeTTa and PeTTa agree exactly on the distinct direct and depth-2 target genes for three representative query SNPs (the raw proof rows dedupe by target gene to these counts):

| Query SNP | direct targets | depth-2 targets | novel |
|---|---|---|---|
| rs10000544 | 9 | 12 | 3 |
| rs10000620 | 5 | 5 | 0 |
| rs10000960 | 6 | 8 | 2 |

The five novel depth-2 targets are:

| Gene | Symbol | Via SNP | Evidence path |
|---|---|---|---|
| ENSG00000150961 | SEC24D | rs10000795 | `(ct cr3 (dt eq37))` |
| ENSG00000172399 | MYOZ2 | rs10000795 | `(ct cr3 (dt eq39))` |
| ENSG00000178636 | MRPL42P1 | rs10000726 | `(ct cr1 (dt eq30))` |
| ENSG00000248394 | FOSL1P1 | rs10001048 | `(ct cr25 (dt eq87))` |
| ENSG00000273077 | (lncRNA) | rs10001048 | `(ct cr25 (dt eq89))` |

The proof terms are machine-checkable evidence chains: `(ct cr3 (dt eq37))` reads "co-regulation transfer via `cr3` (rs10000544↔rs10000795), applied to direct eQTL `eq37` (rs10000795→SEC24D)." On these focused three-query drivers the total wall-clock time is 0.13 s on CeTTa versus 4.27 s on PeTTa. We therefore keep the novel-gene discovery story, but we do *not* reuse the older proof-count/time row as benchmark evidence in this revision.

Biologically, the novel discoveries include MYOZ2 (hypertrophic cardiomyopathy gene, OMIM CMH16) and SEC24D (Cole-Carpenter bone syndrome), both on chr4q26—suggesting a trans-regulatory link from a metabolic locus (PDE5A/FABP2) to cardiac and skeletal gene targets via shared cis-regulatory architecture.

The older "all 7 query SNPs plus reverse queries" timing line is not used as a fresh paper benchmark here; the audited evidence in this revision is the focused cross-runtime driver above.

**Complete benchmark ladder.** The following table summarizes all benchmarks, including both genomic inference (where CeTTa wins) and pure search (where PeTTa wins):

| Benchmark | Scale / note | CeTTa | PeTTa | HE |
|---|---|---|---|---|
| Shallow typed BC | 60 proof witnesses | 4.0 ms avg | 45.4 ms avg | 402 ms avg |
| Translator (`prog1`) | 30 proof witnesses | 3.7 ms avg | 44.8 ms avg | 319 ms avg |
| PLN evidence core | 183 scored pairs | 10.9 ms avg | 58.1 ms avg | 7.56 s |
| Full WM-PLN raw-loader | 1,399,997 lines, 34 proofs / 8 hyps | 16.4 s | 44.5 s | import block |
| Mine→chain focus | 3 query SNPs, 5 novel genes | 0.13 s | 4.27 s | — |
| Deep 17-phase genomic BC | 1,411,430 hypotheses | 34.0 s | 58.1 s | — |
| 8-Queens (direct shared lane) | 92 solutions | 8.22 s | 0.05 s | 2:08.77 |
| Deep proof search | 85,013 proofs | 36.42 s | 7.60 s | >120 s |

*Peak resident memory on the two heavy rows.* On the full WM-PLN raw-loader, CeTTa uses 2.10 GiB against PeTTa's 4.75 GiB (CeTTa lower). On the deep 17-phase genomic BC the direction reverses: CeTTa uses 15.6 GiB against PeTTa's 4.86 GiB—about 3.2× PeTTa's footprint, the memory-multiplicity cost of CeTTa's answer materialization on the high-fan-out row.

CeTTa still wins on several real inference rows, but the revised ladder is intentionally smaller than earlier drafts. In the 2026-06-27 audit we excluded the historical cross-engine miner speed table, the old full-1.4M WM-PLN scaling table, BTree, and the full tilepuzzle row from this cross-engine ladder because at that point the tree either lacked a stable current cross-runtime driver for them or exposed a correctness/comparability issue. (The tilepuzzle regression has since been closed as a single-runtime capacity result by the evaluation-arena garbage collector, Section 7.2; a cross-engine tilepuzzle row has not yet been rerun on the post-GC tree.) The benchmark rows shown here are only those rerun directly in that audit.

### 6.8 Cross-Engine Chaining Suite

To ground the inference-side comparison on community-authored code rather than bespoke benchmarks, we assembled a cross-engine chaining suite that builds matched cross-engine rows from the same chainer families on CeTTa (`-lang he`) and PeTTa, with a per-row cross-engine contract (every result is validated against the other engine's output, never against CeTTa's own output). The sources are Geisweiller's dependent-typed DTL chainers and Metamath `demo0` proofs (github.com/ngeiswei/chaining), Treutlein's `PeTTaChainer` (github.com/rTreutlein/PeTTaChainer), and the PeTTa engine and its PLN library (github.com/patham9/PeTTa). The suite, generators, and driver are in the CeTTa repository under `benchmarks/chaining_crossengine`. In this paper revision we use the suite as audited reproducibility material rather than as an extra source of unevaluated claims: the benchmark rows retained in the paper are only the subset rerun directly in this audit.

**Multi-relational deep chaining.** The strongest current multi-relational row is the 17-phase snapshot `tests/bench_bio_bc_deep_snapshot_20260327.metta`. It extends the mined-pattern chainer with pathway membership, same-pathway transfer, disease associations, compound–disease treatment links, and compound–gene binding links. The full variant-to-drug row is still a typed backward-chaining program:

```
(rule-sig dg (-> (: $e1 (target $snp $gene))
                 (: $e2 (has-disease $gene $disease))
                 (disease-reach $snp $disease $gene)))
(rule-sig td (-> (: $e1 (disease-reach $snp $disease $gene))
                 (: $e2 (treats $compound $disease))
                 (drug-hypothesis $snp $compound $disease $gene)))
```

The current rerun produces exactly **1,411,430 hypotheses** across 17 query phases, with the same totals on both runtimes. The largest buckets are 613,978 `pw-target` hypotheses and 796,964 `deep-pw` hypotheses; the smaller focused buckets are 11 binding hypotheses, 11 disease hypotheses, 210 drug hypotheses, 240 deep-drug hypotheses, and 16 pathway-reach hypotheses. The current timings are **34.0 s on CeTTa** versus 58.1 s on PeTTa (about 1.7× faster). The textbook chain rs10000544 → PDE5A → hypertension ← Sildenafil composes three independent data sources (GTEx, Hetionet, DrugBank) by typed backward chaining with proof terms at every step.

**Dependent binder encoding.** PLN rule declarations use `(rule-sig name type)` instead of `(: name type)` to avoid a current HE spec limitation: the `(: $e T)` syntax in arrow-domain positions is treated as a literal expected type, not a dependent binder, causing `BadArgType` on both HE and CeTTa (see Section 10). The `rule-sig` encoding is semantically equivalent on all runtimes.

## 7 Performance Analysis

### 7.1 Current profiling status

Valgrind reruns on the grounded targets, measured on the pre-GC tree (2026-06-27), split into one success mode and one blocker. Massif reruns succeeded on both the restored 8-Queens row and the full TFBS-inclusive WM-PLN row. On direct 8-Queens, the fresh peak useful heap is 173.5 MB, dominated by arena allocation (`arena_claim_block`), `atom_symbol_id`,

`atom_deep_copy_impl`, and the typed dispatch / match / eval path. On the full WM-PLN row, the fresh peak useful heap is 2.09 GB, dominated first by import-time term-universe growth and symbol interning (`cetta_realloc`, `term_universe_insert_new_record`, `tu_intern_symbol`, `parse_metta_file_ids`), then by query-time delayed result storage (`stree_insert`, `native_query`). By contrast, fresh Callgrind reruns on both grounded targets do *not* yet yield a usable callgraph file: Valgrind hits a `brk segment overflow` before emitting a nonempty callgrind output. We therefore report fresh Massif measurements here, but continue to withhold exact Callgrind percentages on the current build.

### 7.2 Tilepuzzle: regression closed by the evaluation-arena garbage collector

The 8-puzzle breadth-first search (BFS) remains a useful WAM-class stress test. The patched CeTTa port (`tests/test_tilepuzzle.metta`) returns the labeled result `(tilepuzzle-reachable-state-count including-start 181440)`, which matches an independent BFS check of the solved 8-puzzle parity class. The current PeTTa reference example (`examples/tilepuzzle.metta`) self-checks 181,441 states, so that older reference lane is off by one under the clarified contract.

The whole search runs as a single top-level form, and each BFS step builds a fresh evaluation environment. Before the fix, those per-step environments were never reclaimed inside the evaluation arena, so memory grew linearly in the number of visited states and the full run outgrew its time and memory envelope. The fix is a tail-safe garbage collector for the evaluation arena, on by default. It (i) skips environment merging for *closed* queries—queries with no free variables escaping to the caller—and (ii) at tail-call-safe points evacuates the live root set (the current term, its environment, and its evaluation type) into a small survivor arena, then resets the evaluation arena to a per-call anchor; the survivor arena is itself reset at each top-level form boundary. Because only the roots that outlive a step are copied forward, the transient per-step environments are freed in bulk instead of accumulating.

With the collector on (commit 9237afd), the full run visits all 181,440 states in 8.80 s at 233 MiB peak resident memory. Over that run it reclaims 5,704,882,536 bytes (about 5.3 GiB) of transient evaluation-arena memory, while the survivor arena peaks at only 135,432 bytes (about 132 KiB) and is reset once. The result is independent of the collection budget: at a 1 MB collection budget the run returns the identical 181,440-state payload as at the default budget, with peak resident memory even lower at the tighter budget (178 MiB vs. 233 MiB)—evidence that the collector preserves the computed result rather than changing it. The collector is differentially validated, not verified: a collector-on versus collector-off comparison over the fast regression corpus agrees on all 406 cases, and the same comparison under ASan/UBSan with provenance assertions agrees on all 404 cases it runs (2 further cases skipped only because the collector-off reference exceeded its bounded time limit).

### 7.3 Structured Space Kinds

CeTTa now supports three ordered space kinds beyond plain atomspaces:

| Kind | Discipline | Use case |
|---|---|---|
| `queue` | FIFO push/pop | BFS frontiers, work queues |
| `stack` | LIFO push/pop | DFS, continuation stacks |
| `hash` | O(1) membership | Visited sets, deduplication |

These are created via `(new-space queue)`, `(new-space hash)`, etc., and support universal pattern matching through the standard `match` API. The tilepuzzle uses queue-space for its BFS

frontier and hash-space for visited-set deduplication. Peak memory fell from 6.3 GB with a plain atomspace to 4.68 GB with the queue and hash spaces (pre-GC), and then to 233 MiB once the tail-safe evaluation-arena garbage collector also reclaims the transient per-step environments (commit 9237afd; Section 7.2).

### 7.4 Architecture: Three Machines

Profiling and design analysis (informed by GPT-5.4 Pro architectural review) identifies three semantically distinct machines that should not share mutable state:

1. **Local search machine.** Branch-local bindings, trail/rollback, choice frames, scratch arenas. WAM-inspired: cheap undo via watermarks, not clone/free churn. This is the next implementation priority.

2. **Indexed shared-space machine.** Persistent PathMap/MORK-backed spaces, snapshot/epoch discipline, conjunction query planning, leapfrog triejoin. Deterministic and committed—no backtracking through the index.

3. **Type/elaboration machine** (future). Scoped binders, metavariables, constraint solving, delayed assignments. Separate from runtime bindings.

Language profiles (`he_compat`, `he_extended`, `he_prime`, future `petta`) are compositional assemblies over these machines, not ad-hoc forks in the evaluator.

### 7.5 Faithful Backend Obligations

The indexed shared-space machine needs a sharper correctness contract than "the backend is fast and seems to return the same answers." Recent Lean work factors the backend story into three explicit runtime seams:

1. **Candidate selection.** The index narrows candidate atoms, while CeTTa's native matcher performs exact variable-sensitive matching. This is the two-phase architecture now validated by concrete selector theorems over PathMap tries and by a concrete matcher instantiation on the HE side.

2. **Canonical storage.** Storage may quotient results to canonical support, but it must preserve co-reference. Repeated source variables must remain repeated after storage and materialization; distinct variables must remain distinct.

3. **Revision-based caching.** Cached query answers are valid only across mutations that preserve the relevant support boundary. Duplicate adds or high-count decrements may preserve cache validity; first adds and last removes must invalidate. Variant-normalized cache keys must also preserve right-hand-side answers and result shape under materialization.

This yields a practical division of labor for the PathMap/MORK backend:

1. PathMap stores canonical support and performs prefix-based candidate narrowing;

2. the native matcher remains the source of truth for exact bindings and binder hygiene;

3. count metadata, not trie-key distortion, carries bag semantics.

**Verified runtime contracts.** Recent Lean work extends the backend correctness story with three new contract files:

- `CeTTaRuntimeContracts.lean` (20 theorems): canonicalization injectivity and BEq preservation (`term_canon.c`), search-context save/rollback correctness (`search_machine.c`), effect classification safety (`runtime_effect_classes.json`), and profile hierarchy (`session.c`). Each theorem maps to a concrete refactoring risk it reduces.
- `IncrementalTableSemantics.lean` (15 theorems): specifies partial-table soundness, answer monotonicity, and completion for future tabling in `table_store.c`. Any correct implementation must satisfy `TableEntry.Sound` (partial reads are genuine) and `TableEntry.Exact` (completed tables equal `queryEquations`).
- `ProducerChoicePoint.lean` (10 theorems): formalizes the search-machine save/rollback/nested-choice-point protocol with combined bindings+scratch state.

Crucially, CeTTa's imported backend is binding-correct not through trusted direct bridge bindings, but because candidate narrowing is followed by seeded native rematch (`space_subst_match_with_seed`). The theorem `imported_seedRematch_binding_parity` in `SeedRematchContract.lean` formalizes this: when imported candidates are sound, the two-phase query produces the same result set as direct matching.

**Positive example.** Adding an existing atom to a counted space may increment multiplicity without changing trie support or invalidating unrelated cached queries.

**Negative example.** Letting the backend bridge perform full unification over serialized query terms invites exactly the binder-hygiene failures that arise when canonicalization preserves shape but breaks co-reference.

These obligations are now mapped one-for-one to runtime seams: `space_match_backend.c` for candidate selection, `term_universe.c` for canonical storage, and `table_store.c` for cache invalidation. The companion note *Faithful Backends for HE MeTTa* develops the full proof story, including the variant-query and binding-cache lemmas; the present paper uses those results to justify the architecture of CeTTa's indexed shared-space machine.

### 7.6 MM2 Integration

MM2 is a deterministic, priority-scheduled, forward-chaining term rewriting language implemented by MORK. Its programs are not a separate execution layer—they are atoms in MORK-backed spaces, stepped by MeTTa via `step!`. The hybrid interaction requires no special protocol: MeTTa creates spaces, adds data, steps MM2 rules, and reads results through ordinary `match`.

```
!(bind! &ws (new-space mork))
!(add-atom &ws (exec (0 step)
  (, (edge $x $y) (edge $y $z))
  (0 (+ (path $x $z)))))
!(add-atom &ws (edge a b))
!(add-atom &ws (edge b c))
!(step! &ws)
!(match &ws (path $x $y) ($x $y)) ; -> (a c)
```

Three import paths exist: `.mm2` file import (`import!` or `include` into a target space, which auto-enables the MORK backend), inline `add-atom` of exec rules from MeTTa, and compiled `.act` artifact loading via `mork:open-act`.

Because MM2 rules are ordinary atoms, MeTTa can pattern-match over them as data—inspecting priorities, rewriting source/sink templates, or extracting rules for analysis:

```
!(match &ws (exec $p $src $snk) (rule $p))
```

CeTTa's `mm2_lower.c` handles bidirectional surface↔IR lowering (`exec`↔`mm2_exec`, `,`↔`mm2_pattern_and`, etc.), preserving round-trip fidelity between MM2 surface syntax and CeTTa's internal representation.

# 8 The MeTTa Dialect Landscape

## 8.1 Shared Fragment

85% of MeTTa programs use only shared constructs (`let`, `let*`, `match`, `if`, `case`, `bind!`, `add-atom`, `new-space`). These run identically on all three runtimes without translation. The Metamath proof search benchmark is dialect-agnostic.

## 8.2 Translated Constructs

| Direction | Construct | Translation |
|---|---|---|
| HE→PeTTa | `chain` | `let` |
| | `collapse-bind` | `collapse` |
| | `superpose-bind` | `superpose` |
| | `nop X` | `let $fresh X ()` |
| | `function (return X)` | `X` (unwrap) |
| PeTTa→HE | `progn A B` | `let $fresh A B` |
| | `prog1 A B` | `let $r A (let $d B $r)` |
| | `@<` | `<s` |

## 8.3 Semantic Divergence: Constraint Store vs. Independent Scope

The translation preserves semantics on the *pure fragment* (ground terms, or variables scoped by `let*`). A genuine divergence exists for free variables in non-deterministic branches:

```
(= (deduce (And $a $b)) (And (deduce $a) (deduce $b)))
```

When `deduce` returns via `match &self` with a free variable `$x`, HE/CeTTa bind `$x` consistently across both branches of `And`, while PeTTa gives each branch its own copy.

This divergence has a precise theoretical characterization. Saraswat et al. [2] showed that concurrent constraint languages based on Tell-and-Ask over a shared store satisfy a *monotonicity property*: constraints accumulate monotonically, and any reduction possible in a later configuration was already possible in an earlier one (their Proposition 2.1). HE/CeTTa's `match &self` operates on a shared constraint store—bindings produced by one branch propagate to all others via the shared space. PeTTa's Prolog backend evaluates branches with independent variable scopes, closer to what Saraswat et al. identify as the "read-only variable" model of Flat Concurrent Prolog, which they explicitly note does *not* possess the monotonicity property.

**The depth-1 standard form.** Saraswat et al. define a clause to be in *standard form* when each atom in the body is depth-1 (no shared variables between conjunction branches). Nil Geisweiller's typed backward chainer uses exactly this form: `let*` threads variable bindings explicitly between premises, eliminating implicit sharing. This is why the typed chainer works identically on all three runtimes—it stays within the standard-form fragment where the constraint-store and independent-scope semantics coincide.

**Lean formalization.** The `Translatable` and `PureTranslatable` predicates in `HEPeTTaSound.lean` characterize precisely this fragment: atoms where `atomToPattern` succeeds and produces `isMatchCorrect` patterns (no lambda, subst, or collection nodes). The sorry-free proof establishes that equation-matching semantics are preserved under translation on this fragment. Extending the proof to the shared-variable conjunction fragment would require formalizing the monotonic constraint-store semantics of HE's `match &self`—a natural target for Phase 3 of the Lean proof.

# 9 Non-determinism and Mutable State

A fundamental specification gap in MeTTa concerns the interaction of non-deterministic evaluation with mutable space operations. When `let $x (match &db ...) (add-atom &result (stored $x))` executes with multiple match results, what happens to the shared `&result` space?

## 9.1 The Problem

Consider:

```
!(bind! &db (new-space))
!(add-atom &db (item A))
!(add-atom &db (item B))
!(bind! &result (new-space))
!(let $x (match &db (item $y) $y)
   (superpose ((remove-atom &result (stored $x))
               (add-atom &result (stored $x)))))
!(get-atoms &result)
```

HE returns `[]`—all atoms lost. CeTTa returns `[(stored A) (stored B)]`—correct. The root cause: HE interleaves side effects from different non-deterministic branches, so `remove-atom` from one iteration cancels `add-atom` from another. CeTTa evaluates each iteration's body to completion before starting the next. This is the exact pattern used by the Hyperon pattern miner's `build-specialization`, explaining why the miner produces empty results on HE.

## 9.2 Four Possible Semantics

We analyzed four options across eight representative examples (accumulation, logging, constraint checking, search with backtracking, speculative execution, order-dependent processing, accidental non-determinism, and the real miner code):

| Option | Wins | Loses | Character |
|---|---|---|---|
| Sequential (CeTTa) | 6/8 | 1/8 | accumulation-friendly |
| Interleaved (HE) | 0/8 | 4/8 | always worse than sequential |
| Forked (copy-on-write) | 1/8 | 4/8 | search-friendly |
| Illegal (UB) | 1/8 | 5/8 | safest, least ergonomic |

Sequential evaluation loses only on *search with backtracking*, where each branch needs isolated state. Interleaved evaluation is strictly dominated—it provides no advantage over sequential while introducing race conditions on shared mutable state.

### 9.3 `with-space-snapshot`: The Escape Hatch

For the one case where sequential semantics is wrong (search with mutable state), CeTTa provides `with-space-snapshot`:

```
(= (fs-add-to-ss $rb $ss)
   (with-space-snapshot $ss0 $ss
     (match $ss0 (: $x1 $a1)
       (match $ss0 (: $x2 $a2)
         (match $rb (: $f (-> $a1 $a2 $b))
           (add-atom $ss (: ($f $x1 $x2) $b)))))))
```

This pattern, from Geisweiller's propositional forward chainer, reads from a frozen snapshot `$ss0` while writing to the live space `$ss`—the standard *semi-naive* discipline for stratified forward inference. This is a CeTTa extension; HE does not currently implement it.

### 9.4 Proposed Spec Clarification

The MeTTa spec [1] uses sequential Python pseudocode (line 368: a list comprehension over results) that implies sequential side-effect ordering, but does not explicitly require it. We propose:

> *When a non-deterministic expression produces multiple results and those results are bound via `let`/`chain`/equation dispatch, the body for each result is evaluated to completion—including all side effects—before the body for the next result begins. To evaluate branches with isolated state, use `collapse` first, then iterate explicitly.*

A future `snapshot-space` primitive would provide clean copy-on-write semantics for search/backtracking without baking merge semantics into the language core.

## 10 HE′: Dependent Telescope Extension

Both HE and CeTTa treat `(: $e T)` in arrow-domain positions as a literal expected type, not a dependent binder. This means Curry-Howard backward chainers cannot store rule types as `(: dt (-> (: $e (eqtl $snp $gene)) (target $snp $gene)))` in `&self` without triggering `BadArgType`. This is a current HE spec limitation, not a CeTTa bug.

We formalize an explicit extension, HE′ (he_prime), as a *telescope interpretation* of arrow domains. The extension is exactly one function: `splitDomain`, which recognizes `(: $x T)` as a dependent binder instead of a literal. Everything else (matching, bindings, space operations, equation dispatch) is inherited from HE unchanged.

The telescope walk checks arguments left-to-right: each successful type match extends the binding environment, and later domains and the codomain are instantiated under accumulated bindings. This is formalized in Lean (230 lines, 0 sorries) at `HEPrime/Telescope.lean`, with kernel-checked positive and negative examples:

- **Positive:** applying `dt` to `eqtl_rs1_gene1` infers return type `(target rs1 gene1)` via dependent substitution. Binary rule `ct` with cross-domain dependency (`$snp2` bound by first argument, constraining second domain) also kernel-checked.

- **Negative:** mismatched SNP (`rs2` $\neq$ `rs1`) correctly produces `none`. Standard HE telescope parsing treats the same domain as a plain atom, proving the semantics are distinct.

CeTTa implements HE′ as an opt-in profile (`he_prime`), gated by a single flag (`enable_dependent_telescope`). The implementation mirrors the Lean formalization: `split_dependent_domain`, `function_domain_type`, `bind_domain_binder`, and `infer_dependent_application_types` in `eval.c`.

# 11 Related Work

**Concurrent constraint programming.** Saraswat et al. [2] established the theoretical framework for detecting stable properties of networks in concurrent logic programming languages. Their cc(↓,|) language, with Tell/Ask on shared variables over a monotonic constraint store, provides the semantic model that most closely matches HE/CeTTa's `match &self` evaluation. The distinction between monotonic constraint stores and independent-scope evaluation (Flat Concurrent Prolog) precisely characterizes the semantic boundary between HE and PeTTa.

**MeTTa ecosystem.** The MeTTaIL boot compilation pipeline [3] provides a contract-first, Lean-backed compilation path for MeTTa stdlib modules, complementing CeTTa's direct evaluation approach. The Mettapedia formalization project provides the Lean specifications (`EvalSpec`, `MeTTaEval`, `MatchSpec`) that ground both the translation proof and the CeTTa evaluator design. The PLN (Probabilistic Logic Networks) framework uses MeTTa's backward chaining extensively; the typed chainer pattern from Nil Geisweiller's `nilbc.metta` is the standard PLN idiom that works across all three runtimes.

# 12 Conclusion

CeTTa demonstrates that a direct C implementation of the MeTTa evaluator is competitive with PeTTa on inference workloads, but not uniformly faster. In the fresh reruns reported here, CeTTa wins on startup-dominated shallow inference and on several genomic-inference rows (the 183-pair PLN evidence core and the 1,411,430-hypothesis deep genomic backward-chaining row), while PeTTa wins decisively on pure branching search (the 85,013-proof Metamath row). A cross-engine chaining suite (Section 6.8) confirms that CeTTa's shallow advantage is a startup-and-dispatch effect rather than a scaling property. Historical rows whose current drivers are unresolved or no longer the same computation as their old headline—the old miner speed table and BTree—are excluded from the revised benchmark ladder rather than carried forward on trust. Throughout, CeTTa maintains oracle-verified correctness across a 535-test MeTTa regression suite and a 333-case rule-indexed conformance catalog (24/24 spec-rule capacities, all agree) against upstream HE v0.2.10. The sorry-free Lean proof of HE↔PeTTa semantic correspondence (5,400+ lines, 0 sorries), including import commutativity and operational bridges for state and space allocation, establishes that MeTTa's three runtimes are semantically compatible on the shared fragment. The compiled `.act` space format via MORK enables zero-copy indexed loading, demonstrated at the 100k-atom scale, with the large-scale attach path still under repair. On the process-calculus side, the RhoCalc reaction machine's eval-at-COMM semantics now has a witness-free, axiom-clean Lean characterization (Section 5.4): the quiescent run of the canonical payload-evaluation process is exactly its structurally observable outcome set, with no certification witness in the carrier.

Beyond raw performance, CeTTa now treats its own semantics as a first-class object. The profile family makes `he-compat` (literal Rust HE) and the principled `he` line distinct, recorded

semantic products; the reflective modal surface exposes the one-step reduction relation—$\Diamond$, $\Box$, normal-form—to MeTTa programs themselves, aligned with the Lean OSLF derivation; and the small-step rule table makes the first reduction-rule classes named, reflectable, and removable, with rule-removal tests proving the engine depends on the declared rules rather than on duplicate hidden branches. The congruence bug that motivated this weld—an argument-congruence rule silently missing from the one-step relation on three branches at once—is exactly the drift class this architecture eliminates.

Tilepuzzle is settled capacity evidence again. The clarified contract counts 181,440 reachable boards including the start state, and the older PeTTa example's 181,441 self-check is understood as an off-by-one reference expectation. The performance regression that had made the full run outgrow its envelope was root-caused to unbounded evaluation-arena growth and closed by a tail-safe evaluation-arena garbage collector: the full search now completes in 8.80 s at 233 MiB (Section 7.2). Trail-style rollback, bindings builders, and arena mark/reset remain candidates for further improving search-heavy workloads without converting CeTTa into a Prolog engine. The guiding principle: *backtracking should be rare, local, and hidden—not global, ambient, or user-visible.*

**Future work.**

- **Complete HE one-step coverage:** equation matching is table-routed; migrate the special forms (`chain`, `let`, `let*`, `case`, `switch`) one rule class at a time, each with bag-equality and rule-removal gates.
- **`-compile-langdef` codegen:** a generator emitting both the reflectable pack and specialized C for the hot evaluator under one shared source digest, extending the `-compile-stdlib` blob precedent; an evaluate-by-iterating-one-step mode as reference oracle.
- **Small-step / official-HE correspondence:** the `HESmallStep` relation and its OSLF span exist in Lean (proven; zero sorries); the remaining climb is the delimited collapse/simulation theorems connecting it to the fine `mettaHE` instruction machine.
- **Bounded modal operators:** fuel-bounded `reachable-within` ($\Diamond^{\leq k}$) and `invariant-within` ($\Box^{\leq k}$) over the now-faithful one-step relation.
- **Local search machine:** trail/watermark rollback, `BindingsBuilder`, arena mark/reset for scratch atoms. Target: materially improve search-heavy workloads such as tilepuzzle; the state-count contract is settled and the earlier arena-growth regression is closed by the evaluation-arena garbage collector, so this is now an optimization opportunity rather than a correctness or capacity blocker.
- **Indexed shared-space engine:** leapfrog triejoin (from Vandervorst's `trie_synth.py`), conjunction query planning, SpaceEngine boundary (formalized in `SpaceEngineBoundary.lean`: native $\subset$ pathmap $\subset$ mork, with ACT as artifact-only surface; `ACTArtifactBoundary.lean` classifies `open-act`/`load-act!`/`dump!` as storage seams). Target: 10–100$\times$ on large-KB multi-pattern conjunctions.
- **MM2 as MeTTa data (working):** MM2 exec rules are atoms in MORK-backed spaces. MeTTa creates, inspects, rewrites, and steps them through the ordinary space API. `step!` is the execution interface; `.mm2` files import via `include`/`import!`. MeTTa can pattern-match over MM2 code to analyze or transform programs—the meta-language relationship is native.

- **morkl: expert PathMap algebra (future):** Direct access to PathMap's zipper navigation (5,800 lines of Rust), product-zipper Cartesian joins (2,000 lines), and lattice operations (`pjoin`/`pmeet`/`psubtract`/`prestrict`, 1,600 lines). MORK's kernel already implements 15+ sink types (count, sum, head, hash, and, Z3, WASM) and source types (BTM, ACT, equality/inequality constraints). The Lean formalization (46 files, 19K+ lines) proves the three-phase execution protocol, fold-level aggregation, and PathMap lattice correspondence. The gap is FFI exposure and MeTTa surface: the Rust substrate exists, `lib/morkl.metta` does not yet.
- **HO search engine:** Lash-inspired SAT-driven proof search for PLN and type-directed synthesis. PicoSAT as embedded SAT solver. Separate from the evaluator.
- **Translator CLI:** `translate.sh` with `he2petta`/`petta2he`, recursive and bundle modes. Lean proof covers pure fragment + state + spaces (0 sorries).
- **Reaction-semantics formalization:** extend the witness-free eval-at-COMM characterization (Section 5.4) from the single-reaction bag fragment to concurrent and persistent reactions, and relax the hash-set-free channel side condition.
- Full pattern miner on CeTTa end-to-end (∼12× speedup over PeTTa).

# A LeaTTa Cross-Check Appendix

LeaTTa is a Lean 4 MeTTa runtime and proof development that is useful as an independent semantic cross-check for the HE-facing core of CeTTa. The local checkout used for the measurements in this appendix was a clean LeaTTa 1.0.6 tree at `github.com/MesTTo/LeaTTa`, commit `51b4b52` (`2026-06-22`, "prepare 1.0.6 release"). Upstream release notes and the checked-claims book are published at `mestto.github.io/LeaTTa/`.

This appendix records a local scope-and-spot-check run on 2026-06-24; it is a coverage map, not a new benchmark ladder replacing the main paper. The CeTTa side used `-profile he-extended -lang he`. The LeaTTa side used `-oracle` for assertion-bearing files and `-file` for pure loaders. A bulk pass ran every `.metta` file under the CeTTa tree (1255 files) through LeaTTa.

**Coverage, in three tiers.** The raw aggregate—168 of 951 assertion-bearing files passing in full—is misleading on its own, because most of the CeTTa test corpus exercises extension surfaces that LeaTTa, a deliberately minimal HE interpreter, does not implement and is not meant to. The honest decomposition is:

| Tier | What LeaTTa does on it |
|---|---|
| HE core | **Strong.** Of the 21 `he_*` conformance files, 18 pass in full and 2 partially (one loader carries no assertions), for 203/206 assertions. Full passes span symbols, equal-chaining, backchaining, GADTs, dependent types, higher-order functions, grounded basics, type propagation, states, documentation, and PLN truth values. This is the surface that overlaps the HE evaluation semantics. |
| CeTTa extensions | **Out of scope by design.** The bulk of the 783 partial failures call builtins LeaTTa does not provide: MORK-backed spaces (84 files), PathMap (44), `hyperpose` (27), typed `new-space` `hash`/`queue` (12), `import!` (10), `space-set-match-backend`, and the textual Metamath parser. Failing these reflects LeaTTa's minimalism, not a fidelity gap. |
| Genuine gaps | **In-scope divergences.** A float-only numeric model: `(min-atom (5 4 6))` returns `4.000000` rather than an integer, and rationals such as `1/2` raise "only numbers allowed," so the exact integer/rational tower does not correspond. Typed error contours (e.g. `(+ 5 "S")` → `BadArgType`, empty/wrong-arity `car-atom`/`decons-atom`/`min-atom`) are not reproduced; we treat these as accepted divergences. Two `he_b4` assertions touching inert-symbol and empty-match semantics, and the higher-order WM-PLN bridge (218/221 on this checkout), remain to be diagnosed. |

**Spec repair triggered by the cross-check.** The LeaTTa comparison was useful not only as an oracle but as a debugger for our own HE Lean surface. During this pass we found that the public equation-query path had been routed through the simplified `simpleMatch` helper even though the faithful two-sided `matchAtoms`/`mergeBindings` semantics already existed. The repair is now written down in three Lean files: `DeclMatchSpec.lean` gives a declarative specification of `match_atoms` and proves `matchAtoms_sound`/`_complete`; `DeclMergeSpec.lean` does the same for `merge_bindings`, proving `mergeBindings_sound`/`_complete`; and `QuerySurfaceAgreement.lean` proves the staged ground-fragment adequacy theorems `queryEquations_ground_two_way_adequate` and `queryEquationsAgainstVisible_ground_two_way_adequate`. So the cross-check is now anchored on the faithful ground-query surface rather than on the older simplified one. The remaining variable-bearing query case is deliberately left explicit: it requires the next repair, namely threading equalities through `Bindings.resolve`/`Bindings.apply`, and is the right next theorem rather than a hidden assumption.

**Performance.** On matched runs LeaTTa is correct where it is in scope but slower and markedly more memory-hungry than CeTTa's C runtime—operational overhead expected of an executable Lean interpreter, not a semantic limitation.

| Benchmark | CeTTa | LeaTTa | Note |
|---|---|---|---|
| tail_recursion_deep_regression | 0.30 s | 0.80 s | both pass 3/3 assertions |
| tail_recursion_deep | 0.30 s | 0.80 s | both pass 3/3 assertions |
| he_c3_pln_stv | 0.10 s | 0.10 s | both pass 5/5 assertions |
| he_d2_higherfunc | 0.10 s | 0.10 s | both pass 25/25 assertions |
| test_wmpln_ho_similarity_bridge | 0.10 s | 0.10 s | CeTTa passes; LeaTTa passes 218/221 assertions |
| bio_refseq_723k | 0.40 s | 3.30 s | both run; LeaTTa ≈1.16 GB RSS vs. CeTTa ≈196 MB |
| bio_tfbs_415k | 0.30 s | 2.10 s | both run; LeaTTa ≈917 MB RSS vs. CeTTa ≈136 MB |

**Tilepuzzle: surface vs. performance.** The raw CeTTa tilepuzzle benchmark depends on CeTTa's typed `queue` and `hash` spaces, so it does not run unchanged on LeaTTa (a Tier-2 surface mismatch). A bounded adaptation replacing those spaces with a functional queue and plain named-space membership does run: a 100-step witness reaches 171 visited states in 18.47 s, while the 500-step version did not finish under a 30 s cap. For reference, the clarified native contract counts 181,440 reachable states including the start board, while the older PeTTa example's 181,441 self-check is now understood as an off-by-one reference expectation. A fresh full native CeTTa run now completes with the tail-safe evaluation-arena garbage collector: 181,440 states in 8.80 s at 233 MiB (Section 7.2). So this bounded LeaTTa adaptation is only evidence that the surface idea transfers, while the full native CeTTa row is settled capacity evidence.

**Bottom line.** LeaTTa is a credible independent oracle and a candidate verified engine-floor for the HE evaluation core of CeTTa: it agrees on the conformance surface that matters, diverges only on extension builtins it deliberately omits and on a float-only numeric model, and pays an expected interpreter-speed cost. Linking LeaTTa to the HE Lean specification is therefore well-founded on the core, provided the numeric tower and error-contour divergences are encoded as known boundaries. The specification side is now in a better state than when this appendix was first drafted: the ground-query fragment is connected to the faithful HE matcher/merge semantics by proved two-way adequacy, and the remaining variable-bearing query case has been isolated cleanly enough to be attacked next rather than smudged into the oracle story.

# B Experimental — MAM: The MeTTa Abstract Machine

*This appendix is explicitly experimental. It captures design exploration from ongoing discussions (April 2026) between the authors and external reviewers (GPT-5.4 Pro), concerning the post-phase-1f abstract-machine layer for* CeTTa*. Nothing here is yet implemented or committed architecture. It is recorded to preserve design rationale; the authoritative implementation plan remains* `implementation_plan.tex`*.*

## B.1 Motivation

The phase-1f-era extraction seam (`src/search_machine.{h,c}`) pulls policy parsing, stream combinators, and a `TableStore` plugin out of `eval.c`, and wraps them in a thin callback struct `CettaSearchMachine = {stream_source, eval_bound_single, eval_ctx, table_plugin}`, stack-allocated at each call site. This is a useful refactor, but it is a *plug interface* — not an abstract machine in the WAM / ZAM / SLG-WAM sense. Real abstract machines (XSB's

SLG-WAM, SWI's tabling runtime, Soufflé's compiled Datalog, miniKanren's CPS-transformed search) have persistent machine state, explicit visitor protocols, capability advertisement, and episode lifetimes.

The proposal here is a *principled* next layer that retains the current code as `SearchAdapter` (transitional helper) while naming the real abstraction from first principles.

### B.2 Current categorical reading

The MAM's present semantic reading, under active revision, models it as a **quantale-enriched traced profunctor**, fibrated for reflection. This reading is provisional — it captures the current best understanding of the structure, not a definitional claim that binds future architecture. Unpacked:

- **Profunctor** $P : \mathcal{Q}^{\mathrm{op}} \times \mathcal{A} \to \mathcal{V}$: *search.* Queries in $\mathcal{Q}$ relate to answer streams in $\mathcal{A}$, with intensity weighted by elements of $\mathcal{V}$.
- **Quantale enrichment** $\mathcal{V} = (L, \otimes, \leq, \bigvee)$: *multiplicity.* Answer weights live in a quantale: $\mathcal{V} = \mathbf{2}$ for classical boolean search; $\mathcal{V} = \mathbb{N}$ for counted answers; $\mathcal{V} = [0,1]^2$ (strength $\times$ confidence) for PLN. PathMap's trie-algebra is natively a quantale.
- **Trace** $\mathrm{Tr}_X^{A,B} : \mathrm{Hom}(A \otimes X, B \otimes X) \to \mathrm{Hom}(A, B)$: *fixpoint.* Feedback / saturation / tabling / cyclic queries are all instances of categorical trace (Joyal–Street–Verity, Hasegawa).
- **Kleisli structure over an effect monad** (or algebraic effect handlers *à la* Plotkin–Pretnar): *control.* Demand, cut, suspension, resumption, and schedule are composable effect operations.
- **Fibration** $p : \mathcal{T} \to \mathcal{B}$ (optional, for reflective MeTTa): rules-over-machine-state.

A trace operator is provably equivalent to taking the least fixpoint of a parameterised endofunction (Hasegawa 1997); this is why tabling, PLN-convergence, and saturation all fit one uniform operator.

### B.3 The proposed machine family: MAM

We name the machine family **MAM: MeTTa Abstract Machine**. The name is chosen by identity rather than by current structure or scope: MAM is the abstract machine for the MeTTa language family, and that identification remains accurate as the internal architecture evolves. The naming tradition (WAM, ZAM, STG, CEK, CAM) uses either author identity or component decomposition; MAM uses the language identity it serves, which is stable across architectural iteration. Specifically, MAM does *not* hardcode a categorical claim (as "QAM = Quantale Abstract Machine" would have done), and does *not* hardcode a functional scope (as "SCAM = Search & Control Abstract Machine" would have done). The categorical reading and the current feature set are both free to evolve without renaming.

| Concept | Name |
|---|---|
| Machine family | `MAM` (Quantale Abstract Machine) |
| Formal description | Quantale-enriched traced profunctor |
| Engine 1 (goal-directed, Kleisli/effect) | `Chainer` |
| first realization (native) | `native_chainer` |
| Engine 2 (fixpoint, trace) | `Saturator` |
| first realization (pathmap) | `pathmap_saturator` |
| Per-session state | `Episode` |
| Dispatch vtable | `MAMOps` |
| Answer | `Answer` / `WeightedAnswer` |
| Schedule axis | `dfs` / `bfs` / `iddfs` / `interleave` / `native` |
| Demand axis | `unbounded` / `once` / `limit-k` / `fold` |
| Visitor control | `CONTINUE` / `STOP` / `ERROR` / `SUSPEND` |

### B.4 The two-engine pressure strategy

Rather than over-specifying a first concrete machine, MAM is pressured from day 1 by *two* independent realizations that stress complementary axes of the seam:

**`Chainer` (goal-directed).** WAM-lineage control: explicit environments/frames, choicepoints, trail/undo, deterministic tail-path handling, bounded-demand response, direct conjunction when advertised. Design-guide workload: **metamath backward chaining**. Before the evaluation-arena garbage collector this workload could exhaust memory on CeTTa; with the collector CeTTa now completes the Metamath proof-search rows (Section 6), so the remaining motivation here is WAM-lineage control, not a memory-exhaustion gap. First realization: `native_chainer`.

**`Saturator` (fixpoint).** SLG / Datalog / saturation lineage: snapshots, delta-sets, conjunction planning, exact membership, counted answers, eventual path toward PLN and ATP-style saturation. Design-guide workload: **PLN multi-pattern deduction**. First realization: `pathmap_saturator` (because PathMap's quantale structure makes weighted answer aggregation natural).

This pair creates the right kind of pressure on the seam: Chainer forces frames, choicepoints, `EnvRef`, demand handling, and explicit scheduling; Saturator forces snapshots, delta sets, conjunction planning, exact membership, counted answers, and the eventual path to PLN and ATP.

Compiled / bytecode (ZAM-lineage) and ATP saturation engines are explicit *later* lanes — they risk freezing the protocol in the wrong shape if pursued before Chainer and Saturator coexist cleanly under one MAM.

### B.5 Axes that must stay separate

A core architectural commitment is that the following axes are **distinct data** in `SearchRequest`, not tangled behind surface-level operators:

**Schedule.** `dfs` / `bfs` / `iddfs` /`interleave` are *fairness policies*, not engines. Korf's classic result motivates `iddfs` as a completeness-friendly default over raw `bfs` (BFS completeness with DFS memory); miniKanren's interleaving exists precisely to avoid starvation on infinite branches.

**Demand.** `once` / `limit-k` / `unbounded` /`fold` describes how many answers are requested, independently of schedule. `once` and `select 1` must build `demand = FIRST`; they must *not* silently hardcode DFS.

**Table mode.** `none` / `variant` / `subsumptive` /`incremental` mirrors the XSB / SWI ladder; each is an explicit engine-level choice.

**Backend.** `native` / PathMap / MORK chosen by the actual `Space`, not by a global preference.

**Lane.** `recursive-dependent-proof` / `atp-saturation` /`solver-oracle` / ... chosen at surface.

The request builder (one per `-lang`) normalizes surface syntax into `SearchRequest + SearchLangContract`; the MAM consumes the normalized object, never raw surface atoms.

### B.6 The seven-type vocabulary

Seven types are proposed to land together before the first real backend optimization (counted-answer variant table, PathMap-native conjunction, native exact-match index). Landing them simultaneously prevents the "first optimization forces a rewrite" pathology.

| Type | Categorical role |
|---|---|
| `SearchRequest` | Object of $\mathcal{Q}$ |
| `SearchAnswer` | Object of $\mathcal{A}$ (quantale-weighted) |
| `SearchLangContract` | Subcategory inclusion $\mathcal{C} \hookrightarrow$ MAM |
| `SearchCaps` | Capabilities = presence of morphisms/natural transformations |
| `VisitCtl` | Algebra of the effect operation visit |
| `MAMOps` | Kleisli dispatch vtable |
| `Episode` | One trace-iteration instance |

Implementation-actual `SUSPEND/RESUME` behavior, cancellation tokens, snapshot-token machinery, and per-`-lang` registration remain explicitly deferred.

### B.7 Reference base

The proposal draws on the following literature, all available in the local library:

- *Aït-Kaci 1991, WAM Tutorial Reconstruction* – foundational WAM semantics; drives `Chainer` design.
- *Swift & Warren, XSB System Manual; Sagonas et al. 1994* – SLG-WAM tabling; drives `Saturator` / table-mode ladder.
- *Scholz, Jordan, Subotić, Westmann 2016, Soufflé: On Fast Large-Scale Program Analysis in Datalog (CC)* – staged compilation of fixpoint computation; compiled `Saturator` path.
- *Keoliya & Byrd 2020, microKanren* – interleaving search as a distinct schedule axis.
- *Ager, Biernacki, Danvy, Midtgaard 2003, A Functional Correspondence Between Evaluators and Abstract Machines (BRICS)* – derivation methodology for machines from evaluator semantics.

- *Leroy 1990, The ZINC Experiment* – machine-description vs. run-instance separation (maps to `MAMOps` / `MAM` / `Episode`).
- *Russell & Norvig, AIMA Ch. 3* – IDDFS completeness; uniform treatment of uninformed search.

### B.8 Status

This appendix records design rationale; it does not commit implementation. Reasoned design review converged on ∼0.88 confidence on MAM naming and two-engine strategy. The concrete implementation plan (phase 2 seam; phase 3 tabling plugin; phase 4 backend parity) is the authoritative schedule and is unchanged by this appendix.

*This text should be revisited at each phase boundary and revised as implementation reveals which proposals held and which did not.*

## References


[1] B. Goertzel et al., "MeTTa: A Reflective Language for AGI," OpenCog Foundation, 2024.

[2] V. Saraswat, D. Weinbaum, K. Kahn, and E. Shapiro, "Detecting Stable Properties of Networks in Concurrent Logic Programming Languages," in *Proc. 7th ACM Symposium on Principles of Distributed Computing (PODC)*, pp. 210–222, 1988.

[3] Z. Goertzel and Oruži, "MeTTaIL: Contract-First Boot Compilation for Direct C MeTTa Runtimes," Working paper, 2026.

[4] L. G. Meredith and M. Radestock, "A Reflective Higher-Order Calculus," *Electronic Notes in Theoretical Computer Science*, vol. 141, no. 5, pp. 49–67, 2005.

[5] L. G. Meredith, "Continued Interactive GSLTs and the Cost Endofunctor: Wrapping by Construction," Working paper, 2026.